L3: Introduction to SQL

Md. Tariquzzaman
Junior Lecturer, Dept. of Computer Science and Engineering
Islamic University of Technology (IUT)

CSE 4307: Database Management Systems

Contents

3.1 Overview of the SQL Query Language

3.2 SQL Data Definition

3.3 Basic Structure of SQL Queries

3.4 Additional Basic Operations

3.5 Set Operations

3.6 Null Values

3.7 Aggregate Functions

3.8 Nested Subqueries

3.9 Modification of the Database

3.1 Overview of the SQL Query Language

Understanding SQL Functionalities

- **Data Manipulation Language (DML)** Think of the database as a big filing cabinet. DML lets you:
  - **Insert** new files (add data),
  - **Delete** unwanted files (remove data),
  - **Update** existing files (change data), and
  - **Query** information (search or read data).

  It's like the "hands" that help you work with the data inside your cabinet!

- **Integrity Constraints** These act like "rules" or "locks" that keep your data accurate and meaningful. For example, you can't store a student's negative age! DDL (Data Definition Language) is used to set up such integrity conditions.

- **View Definition** Imagine a "view" as a custom window into the database — you decide what part of the data you want to see. The DDL helps create these virtual tables for easier access and security.

Understanding SQL Functionalities

- **Transaction Control** Think of a transaction as a "to-do list" for your database — either all tasks succeed, or none do (like an all-or-nothing deal). Commands like `BEGIN`, `COMMIT`, and `ROLLBACK` help manage the start, success, or undoing of transactions.
- **Embedded SQL and Dynamic SQL** Imagine mixing SQL with your favorite programming language (like C, Java, or Python). Embedded SQL lets your code talk directly to the database. Dynamic SQL, on the other hand, can build SQL commands on the fly — like a chef deciding the recipe while cooking!
- **Authorization** This defines "who can do what." It's like setting permissions on your Google Drive — some can view, some can edit, and others can't access at all. SQL authorization commands protect your data from unwanted access.

3.2 SQL Data Definition

3.2 SQL Data Definition — Overview

- SQL uses the **Data Definition Language (DDL)** to describe the structure of a database.
- DDL defines:
  - What **relations (tables)** exist,
  - What each relation contains (its **schema**),
  - What **rules, constraints, and access rights** apply,
  - How data is **stored** on disk.
- Think of DDL as the architect's blueprint — it plans the structure before data moves in.

What Does DDL Specify?

- **Schema:** The structure of each relation (table).
- **Attribute Types:** The kind of data each column can hold.
- **Integrity Constraints:** Rules to keep data correct.
- **Indices:** Help speed up data access.
- **Security & Authorization:** Controls who can access or modify data.
- **Physical Storage:** How data is arranged on disk.

3.2.1 Domain Types — SQL's Building Blocks

- **char(n):** Fixed-length character string (always n chars). Example: `char(5)` stores "CS" as "CS ".
- **varchar(n):** Variable-length character string (up to n chars). Example: `varchar(20)` stores "Alice" as exactly 5 characters.
- **int:** Integer — machine-dependent range.
- **smallint:** Smaller integer — efficient for small numeric values.
- **numeric(p, d):** Fixed-point number with total p digits and d digits after the decimal. Example: `numeric(3,1)` allows 44.5 but not 444.5 or 0.32.
- **real, double precision:** Floating-point and double-precision numbers — machine dependent.
- **float(n):** Floating-point number with at least n digits of precision.

Note

More advanced data types are discussed in Chapter 4.

char vs. varchar — The Subtle Difference

- **char(n)** → Fixed size: fills with spaces if text is shorter. Example: "Avi" in `char(10)` = "Avi "
- **varchar(n)** → Flexible: stores only the actual text length.
- When compared:
  - Some DBs ignore trailing spaces,
  - Others treat "Avi" and "Avi " as different.
- **Tip:** Use `varchar` instead of `char` to avoid comparison issues.
- Also: `nvarchar` supports multilingual (Unicode) data.

3.2.2 Basic Schema Definition — Creating Tables

- The **CREATE TABLE** command defines a new relation.

Example

```
create table department (
 dept_name varchar(20),
 building varchar(15),
 budget numeric(12,2),
 primary key (dept_name));
```

- Creates a table named `department` with 3 columns.
- `dept_name` is the **Primary Key** — must be unique and not null.

General Form of CREATE TABLE

Syntax

```
create table r (
 A1 D1, A2 D2, ..., An Dn,
 <integrity_constraint1>, ..., <integrity_constraintk>);
```

- r = table name
- A_i = attribute name
- D_i = data type or domain
- Optional integrity constraints maintain data validity.

Integrity Constraints in SQL

- **Primary Key:** Ensures each record is unique and not null. Example: `primary key (dept_name)`
- **Foreign Key:** Links one table to another. Example: `foreign key (dept_name) references department(dept_name)` → Ensures a course's department must exist in the department table.
- **Not Null:** Prevents missing values. Example: Instructor name cannot be null.

Integrity Constraints in SQL

- **Primary Key:** Ensures uniqueness and non-null. Example: `primary key (dept_name)`
- **Foreign Key:** Ensures referential integrity — a link must exist in another table. Example: `foreign key (dept_name) references department(dept_name)`
- **Not Null:** Prevents missing values. Example: Instructor name cannot be null.

Example

```
create table instructor (
 ID char(5),
 name varchar(20) not null,
 dept_name varchar(20),
 salary numeric(8,2),
 primary key (ID),
 foreign key (dept_name) references department);
```

How SQL Enforces Constraints

- SQL automatically blocks updates that violate constraints.
- Examples:
  - Duplicate or null primary keys $\rightarrow$ **Error**
  - Invalid foreign key (e.g., non-existent department) $\rightarrow$ **Error**
- Ensures **data consistency and reliability**.

Modifying, Deleting, and Updating Tables

- **Insert:** `insert into instructor values ('10211', 'Smith', 'Biology', 66000);`
- **Delete:** Remove all tuples from a table: `delete from student;`
- **Drop Table:** `drop table r;` — removes both schema and data.
- **Alter Table:** `alter table r add A D;` Adds a new column (all existing tuples get `NULL`).
  `alter table r drop A;` Removes a column.

SQL data definition for part of the university database.

```
create table department
    (dept_name    varchar (20),
     building     varchar (15),
     budget       numeric (12,2),
     primary key (dept_name));

create table course
    (course_id    varchar (7),
     title        varchar (50),
     dept_name    varchar (20),
     credits      numeric (2,0),
     primary key (course_id),
     foreign key (dept_name) references department);

create table instructor
    (ID           varchar (5),
     name         varchar (20) not null,
     dept_name    varchar (20),
     salary       numeric (8,2),
     primary key (ID),
     foreign key (dept_name) references department);

create table section
    (course_id    varchar (8),
     sec_id       varchar (8),
     semester     varchar (6),
     year         numeric (4,0),
     building     varchar (15),
     room_number  varchar (7),
     time_slot_id varchar (4),
     primary key (course_id, sec_id, semester, year),
     foreign key (course_id) references course);

create table teaches
    (ID           varchar (5),
     course_id    varchar (8),
     sec_id       varchar (8),
     semester     varchar (6),
     year         numeric (4,0),
     primary key (ID, course_id, sec_id, semester, year),
     foreign key (course_id, sec_id, semester, year) references section,
     foreign key (ID) references instructor);
```

3.3 Basic Structure of SQL Queries

Basic Query Structure

- A typical SQL query has the form:

```
select A1, A2, ..., An
from r1, r2, ..., rm
where P
```

- A_i represents an attribute
- R_i represents a relation
- P is a predicate
- The result of an SQL query is a relation

The select Clause

- The `select` clause lists the attributes desired in the result of a query
- Corresponds to the projection operation of the relational algebra
- Example: find the names of all instructors:

```
select name
from instructor
```

- NOTE: SQL names are case insensitive
  - E.g., Name $\equiv$ NAME $\equiv$ name

The select Clause - Example with Results

Query: `select name from instructor`

Table: Instructor table

ID	name	dept_name
10101	Srinivasan	Comp. Sci.
12121	Wu	Finance
15151	Mozart	Music
22222	Einstein	Physics

Table: Result table

name
Srinivasan
Wu
Mozart
Einstein

The select Clause (Cont.)

- SQL allows duplicates in relations as well as in query results
- To force the elimination of duplicates, insert the keyword `distinct` after `select`
- Find the department names of all instructors, and remove duplicates

```
select distinct dept_name
from instructor
```

- The keyword `all` specifies that duplicates should not be removed

```
select all dept_name
from instructor
```

distinct vs all - Example with Results

Table: Instructor table

name	dept_name
Srinivasan	Comp. Sci.
Wu	Finance
Mozart	Music
Einstein	Physics
El Said	Comp. Sci.

Query:

```
select distinct dept_name
from instructor
```

Table: Result table (select distinct)

dept_name
Comp. Sci.
Finance
Music
Physics

Table: Result table (select all)

dept_name
Comp. Sci.
Finance
Music
Physics
Comp. Sci.

The select Clause (Cont.)

- An asterisk in the `select` clause denotes "all attributes"

  ```
  select * from instructor
  ```

- An attribute can be a literal with no `from` clause

  ```
  select '437'
  ```

  - Results is a table with one column and a single row with value "437"
  - Can give the column a name using: `select '437' as FOO`

- An attribute can be a literal with `from` clause

  ```
  select 'A' from instructor
  ```

  - Result is a table with one column and N rows, each row with value "A"

The select Clause (Cont.)

- The `select` clause can contain arithmetic expressions involving +, -, *, /, and operating on constants or attributes of tuples
- The query:

  ```
  select ID, name, salary/12
  from instructor
  ```

  would return a relation with the salary divided by 12
- Can rename "salary/12" using the `as` clause:

  ```
  select ID, name, salary/12 as monthly_salary
  ```

Arithmetic Expressions - Example with Results

Query: `select ID, name, salary/12 as monthly_salary from instructor`

Table: Instructor table

ID	name	salary
10101	Srinivasan	65000
12121	Wu	90000
15151	Mozart	40000

Table: Result table

ID	name	monthly_salary
10101	Srinivasan	5416.67
12121	Wu	7500.00
15151	Mozart	3333.33

The where Clause

- The `where` clause specifies conditions that the result must satisfy
- Corresponds to the selection predicate of the relational algebra
- To find all instructors in Comp. Sci. dept:

```
select name
from instructor
where dept_name = 'Comp. Sci.'
```

- SQL allows logical connectives: `and`, `or`, `not`
- Operands can be expressions involving <, <=, >, >=, =, <>
- Example: all instructors in Comp. Sci. with salary > 70000

```
select name from instructor
where dept_name = 'Comp. Sci.' and salary > 70000
```

The where Clause - Example with Results

Query: `select name from instructor where dept_name = 'Comp. Sci.'`

Table: Instructor table

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	Comp. Sci.	60000

Table: Result table

name
Srinivasan
El Said

The from Clause

- The `from` clause lists the relations involved in the query
- Corresponds to the Cartesian product operation
- Example: find Cartesian product instructor $\times$ teaches

```
select * from instructor, teaches
```

- Generates every possible instructor–teaches pair
- Common attributes (e.g., ID) are renamed using the relation name: instructor.ID
- Cartesian product is useful with a `where`-clause condition

Cartesian Product - Example (Partial)

Query: `select * from instructor, teaches`

Table: Instructor table

ID	name	dept	salary
10101	Srinivasan	CS	65000
12121	Wu	Fin	90000

Table: Teaches table

ID	course_id	sem
10101	CS-101	Fall
10101	CS-315	Spring
12121	FIN-201	Spring

Table: Result table (partial)

i.ID	name	dept	sal	t.ID	c
10101	Srin.	CS	65000	10101	C
10101	Srin.	CS	65000	10101	C
10101	Srin.	CS	65000	12121	F
12121	Wu	Fin	90000	10101	C
...	...	...	...	...	.

Examples

- Find the names of all instructors who have taught some course and the course_id

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID
```

- Find the names of all instructors in the Art department who have taught some course and the course_id

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID
and instructor.dept_name = 'Art'
```

Join Example - Results

Query: `select name, course_id from instructor, teaches where instructor.ID = teaches.ID`

Table: Instructor table

ID	name
10101	Srinivasan
12121	Wu
15151	Mozart

Table: Teaches table

ID	course_id
10101	CS-101
10101	CS-315
12121	FIN-201

Table: Result table

name	course_id
Srinivasan	CS-101
Srinivasan	CS-315
Wu	FIN-201

Self Join - Example with Results

Query: `select distinct T.name from instructor as T, instructor as S where T.salary > S.salary and S.dept_name = 'Comp. Sci.'`

Table: Instructor table

name	dept_name	salary
Srinivasan	Comp. Sci.	65000
Wu	Finance	90000
Mozart	Music	40000
Einstein	Physics	95000
El Said	Comp. Sci.	60000

Table: Result table

name
Wu
Einstein
Srinivasan

(All instructors with salary > any Comp. Sci. instructor)

Self Join Example - Hierarchy

Scenario: Find each employee and their supervisor's supervisor (i.e., 2-level hierarchy).

Table: emp-super relation

person	supervisor
Bob	Alice
Mary	Susan
Alice	David
David	Mary

Self Join Example - Hierarchy

SQL Query (Self Join):

```
select E1.person as employee,
E2.supervisor as super_supervisor
from emp_super as E1,
emp_super as E2
where E1.supervisor = E2.person;
```

Result:

Table: Employee → Supervisor's Supervisor

employee	super_supervisor
Bob	David
Mary	NULL
Alice	Mary
David	Susan

3.4 Additional Basic Operations

3.4 Additional Basic Operations

- The Rename Operation
- String Operations
- Attribute Specification in Select Clause
- Ordering the Display of Tuples
- Where-Clause Predicates

3.4.1 The Rename Operation - Basic Query

Query:

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID;
```

Result:

name	course_id
Srinivasan	CS-101
Srinivasan	CS-315
Wu	FIN-201
Mozart	MU-199
Einstein	PHY-101
...	...

Attribute names derived from the relations in the FROM clause

The Rename Operation

- SQL allows renaming relations and attributes using `as`: `old-name as new-name`
- Example: find names of all instructors with salary higher than any Comp. Sci. instructor

```
select distinct T.name
from instructor as T, instructor as S
where T.salary > S.salary
and S.dept_name = 'Comp. Sci.'
```

- Keyword `as` is optional: `instructor as T` ≡ `instructor T`

Need for Renaming

Three scenarios requiring attribute renaming:

1. **Duplicate names:** Two relations may have attributes with the same name
2. **Arithmetic expressions:** Computed attributes don't have names
3. **Custom naming:** May want to change attribute names for clarity

Solution: SQL's `AS` clause

```
old-name AS new-name
```

Renaming Attributes with AS Clause

Without Renaming:

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID;
```

Result columns:

name	course_id

With Renaming:

```
select name as instructor_name,
course_id
from instructor, teaches
where instructor.ID = teaches.ID;
```

Result columns:

instructor_name	course_id

Renaming Relations - Shortened Names

Original Query:

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID;
```

Using Aliases:

```
select T.name, S.course_id
from instructor as T, teaches as S
where T.ID = S.ID;
```

Benefit: Shorter, more convenient names (T, S instead of instructor, teaches)

Self-Join Example - Salary Comparison

Query: "Find names of instructors whose salary is greater than at least one instructor in Biology"

SQL Query:

```
select distinct T.name
from instructor as T,
instructor as S
where T.salary > S.salary
and S.dept_name = 'Biology';
```

T and S are *aliases* (correlation names) for instructor

Sample Result:

name
Wu
Einstein
Brandt
Katz
...

Returns instructors earning more than the lowest-paid Biology instructor

Note 3.1: Sets vs Multisets - The Key Difference

Traditional Relational Algebra (Sets):

- No duplicates allowed
- Each tuple appears once
- Like mathematical sets: {1, 2, 3}

Example:

name
Alice
Bob

SQL (Multisets):

- Duplicates allowed
- Same tuple can appear multiple times
- Like bags: {1, 1, 2, 3, 3, 3}

Example:

name
Alice
Alice
Bob

Why does this matter? SQL keeps track of duplicate counts throughout operations

Multiset Operation 1: Selection (Filtering)

Rule: Duplicates are preserved when filtering

Original table (3 tuples):

name	salary
Alice	50000
Bob	60000
Alice	50000

Notice: Alice appears twice

After: WHERE salary $\geq$ 50000

name	salary
Alice	50000
Bob	60000
Alice	50000

Still 3 rows: both Alice copies kept

Key point: If a tuple appears c_1 times and satisfies the condition, it appears c_1 times in result

Multiset Operation 2: Projection (Selecting Columns)

Rule: Each input copy produces one output copy

Original table:

name	salary
Alice	50000
Bob	60000
Alice	50000

3 rows total

After: SELECT name

name
Alice
Bob
Alice

Still 3 rows: duplicates preserved

Key point: For each row in input, we get exactly one row in output (even if they become identical)

Multiset Operation 3: Cartesian Product (Joining)

Rule: Duplicate counts multiply together

Table r1:

A
x
x

x appears 2 times

Table r2:

B
1
1
1

1 appears 3 times

Cartesian Product r1 × r2:

A	B
x	1
x	1
x	1
x	1
x	1
x	1

(x, 1) appears 2 × 3 = 6 times

Key point: Combine each copy from r1 with each copy from r2

Complete Example: Step-by-Step

Step 1: Given tables

```
r1(A, B):
(1, a)
(2, a)
```

```
r2(C):
(2)
(3)
(3)
```

Note: 'a' appears 2 times
Note: (3) appears 2 times

Step 2: Project column B from r1

```
Result of projection:
(a)
(a)
```

Still 2 copies of 'a'

Step 3: Cartesian product

```
Final result:
(a, 2)  -- 2 × 1 = 2 copies
(a, 2)
(a, 3)  -- 2 × 2 = 4 copies
(a, 3)
(a, 3)
(a, 3)
```

How SQL Maps to Relational Algebra

Every SQL query can be translated to relational algebra operations

SQL Query Format:

```
SELECT A1, A2, ...
FROM r1, r2, ...
WHERE condition
```

What each part does:

- FROM: Creates combinations
- WHERE: Filters rows
- SELECT: Picks columns

Relational Algebra:

$$\Pi_{A_1,A_2,\ldots}(\sigma_{condition}(r_1 \times r_2 \times \ldots))$$

Order of operations:

1. $\times$: Cartesian product (FROM)
2. σ : Selection (WHERE)
3. Π : Projection (SELECT)

SQL to Relational Algebra Mapping

Basic SQL Query:

$$\begin{gathered}\texttt{SELECT}\ \ A_1, A_2, ..., A_n \\ \texttt{FROM}\ \ r_1, r_2, ..., r_m \\ \texttt{WHERE P}\end{gathered}$$

Equivalent Multiset Relational Algebra:

$$\Pi_{A_1, A_2, ..., A_n}(\sigma_P(r_1 \times r_2 \times ... \times r_m))$$

Note: SQL's WHERE clause = Relational algebra's selection (σ)
SQL's SELECT clause = Relational algebra's projection (Π)

3.4.2 String Operations - Basic Syntax

String Specification:

- Enclosed in single quotes: `'Computer'`
- Single quote in string: use two single quotes: `'It''s right'`

Case Sensitivity:

SQL Standard:

```
'comp. sci.' = 'Comp. Sci.'
```

Result: **FALSE**

MySQL/SQL Server:

```
'comp. sci.' = 'Comp. Sci.'
```

Result: **TRUE** (default)

String Functions Overview

Common SQL String Functions:

- `||` - Concatenation
- `upper(s)` - Convert to uppercase
- `lower(s)` - Convert to lowercase
- `trim(s)` - Remove trailing spaces
- Extract substrings
- Find string length

Note: Exact functions vary by database system

Pattern Matching with LIKE Operator

Special Characters:

- % - Matches any substring
- _ - Matches any single character

Pattern Examples:

```
'Intro%'
matches: 'Intro to CS'

'%Comp%'
matches: 'Intro to Computer'
matches: 'Computational Bio'

'___'
matches: any 3 chars

'___%'
matches: 3+ chars
```

Query Example:

```
select dept_name
from department
where building like '%Watson%';
```

Result:

dept_name
Comp. Sci.
Physics

LIKE with Escape Characters

Pattern with Literal %:

```
like 'ab\%cd%' escape '\'
```

Matches strings beginning with ”ab%cd”

Pattern with Literal \:

```
like 'ab\\cd%' escape '\'
```

Matches strings beginning with ”ab\cd”

NOT LIKE:

```
select dept_name
from department
where building not like '%Watson%';
```

Finds departments NOT in Watson buildings

Advanced (PostgreSQL):

```
similar to
```

Supports Unix regular expressions

Positional Pattern Matching

Second Character Match:

```
LIKE '_a%'
```

matches strings where the **2nd** letter is 'a'.

Matches:

- 'Data'
- 'Banana'
- 'Cat'

Non-Matches:

- 'Apple' (2nd is 'p')
- 'A' (too short)

Third Character Match:

```
select name
from instructor
where name like '__n%';
```

matches if **3rd** letter is 'n'.

Result:

name
Einstein
Ken *(Exactly 3 chars)*

Start and End Combinations

Prefix & Suffix:

```
LIKE 'C%e'
```

Starts with 'C', ends with 'e', anything in between.

Matches:

- 'Comp Science'
- 'Code'
- 'Ce' (0 chars in middle)

Query Example:

```
select course_id
from course
where title like 'Intro%Sys%';
```

Finds "Intro" followed eventually by "Sys".

Result:

title
Intro to Systems
Intro to DB Sys

Pattern Matching for Codes/IDs

Fixed Format (Phone/SSN):

```
LIKE '___-___-____'
```

Strictly enforces the format: 3 chars, hyphen, 3 chars, hyphen, 4 chars.

Matches:

- '555-010-9999'
- 'ABC-123-XYZW'

Course Code Logic:

```
select course_id
from course
where course_id like 'CS-1__';
```

Finds all 100-level CS courses (CS-100 to CS-199).

Result:

course_id
CS-101
CS-190

Filtering via Negation and Minimums

Minimum Length Check:

```
LIKE '_____%'
```

Requires string to be at least 5 characters long.

Matches:

- 'Music' (5)
- 'Biology' (7)

Non-Matches:

- 'Bio' (3)

Excluding Patterns:

```
select name
from instructor
where name not like 'A%';
```

Filters out anyone whose name starts with 'A'.

Result:

name
Mozart
Wu

3.4.3 Attribute Specification - Using Asterisk

Select Specific Attributes:

```
select name, salary
from instructor;
```

Result:

name	salary
Srinivasan	65000
Wu	90000

Select All Attributes:

```
select *
from instructor;
```

Result:

ID	name	dept	salary
10101	Srinivasan	Comp. Sci.	65000
83821	Wu	Finance	90000

Selecting from Multiple Tables

When you join tables, both may contain overlapping columns.

```
select *
from instructor, teaches
where instructor.ID = teaches.ID;
```

What happens?

- All columns from `instructor` and `teaches` are returned.
- Both have an `ID` column → duplicate names!

Qualified Asterisk: Selecting from One Table

To get only the columns of one specific table, use a **qualified asterisk:**

```
select instructor.*
from instructor, teaches
where instructor.ID = teaches.ID;
```

Meaning: Return only attributes from `instructor`, ignore columns from `teaches`.

Example: Effect of Using `instructor.*`

Before: Using `select *`

Output of: select *				
ID	name	dept_name	salary	course_id
10101	Srinivasan	Comp. Sci.	65000	CS-101
83821	Wu	Finance	90000	FIN-201

Includes all columns from both tables.

After: Using `select instructor.*`

Output of: instructor.*			
ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
83821	Wu	Finance	90000

Only instructor attributes retained.

3.4.4 Ordering - Single Attribute

Without ORDER BY:

```
select name
from instructor
where dept_name = 'Physics';
```

Result (unordered):

name
Gold
Einstein

With ORDER BY:

```
select name
from instructor
where dept_name = 'Physics'
order by name asc;
```

Result (alphabetical):

name
Einstein
Gold

Default: Ascending order (ASC)

Ordering by a Single Attribute (DESC)

Query:

```
SELECT name, dept_name, salary
FROM instructor
ORDER BY salary DESC;
```

Explanation:

- Sort rows by `salary` in descending order.
- Highest salary appears first.

Result:

Name	Dept	Salary
Katz	Comp. Sci.	95000
Brandt	History	92000
Wu	Finance	90000
Kim	Comp. Sci.	90000
Singh	Physics	88000

Ordering by Multiple Attributes (DESC + ASC)

Query:

```
SELECT name, dept_name, salary
FROM instructor
ORDER BY salary DESC, name ASC;
```

Explanation:

- Primary sort: `salary` (descending).
- Secondary sort: `name` (ascending) when salaries are equal.
- Example: Both Kim and Wu earn 90000 $\rightarrow$ Kim comes before Wu alphabetically.

Result:

Name	Dept	Salary
Katz	Comp. Sci.	95000
Brandt	History	92000
Kim	Comp. Sci.	90000
Wu	Finance	90000
Singh	Physics	88000

Tie on salary $\rightarrow$ sorted alphabetically by name.

3.4.5 BETWEEN Comparison Operator

Using AND:

```
select name
from instructor
where salary <= 100000
and salary >= 90000;
```

Using BETWEEN:

```
select name
from instructor
where salary between 90000
and 100000;
```

Both produce same result:

name
Wu
Katz
Brandt

Also available: `NOT BETWEEN`

Row Constructor - Tuple Comparison

Row Constructor Syntax:

$$(v_1, v_2, \ldots, v_n)$$

Lexicographic Ordering:

- $(a_1, a_2) <= (b_1, b_2)$ is true if $a_1 <= b_1$ and $a_2 <= b_2$
- Tuples equal if all attributes equal

Row Constructor Example - Biology Department Query

Without Row Constructor:

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID
and dept_name = 'Biology';
```

With Row Constructor:

```
select name, course_id
from instructor, teaches
where (instructor.ID, dept_name) =
(teaches.ID, 'Biology');
```

Both produce identical results:

name	course_id
Crick	BIO-101
Crick	BIO-301

Summary - Additional Basic Operations

Key SQL Operations Covered:

- **AS clause:** Rename attributes and relations
- **String operations:** LIKE, pattern matching, functions
- **Asterisk (*):** Select all attributes
- **ORDER BY:** Sort results (ASC/DESC)
- **BETWEEN:** Range comparisons
- **Row constructors:** Tuple comparisons

These operations enhance SQL's expressiveness and make queries more concise and readable.

3.5 Set Operations

What Are Set Operations?

The Big Picture

SQL set operations combine results from multiple queries, just like mathematical set operations combine sets.

- **UNION** ($\cup$): Combine all items from both sets
- **INTERSECT** ($\cap$): Find common items in both sets
- **EXCEPT** ($-$): Items in first set but not in second

Key Analogy

Think of these like combining guest lists for parties:

- UNION: Everyone invited to *either* party
- INTERSECT: People invited to *both* parties
- EXCEPT: People invited to party A but *not* party B

Our Example Data

The Section Table

We'll work with course sections from different semesters.

course_id	semester	year
CS-101	Fall	2017
CS-347	Fall	2017
PHY-101	Fall	2017
CS-101	Spring	2018
CS-319	Spring	2018
CS-319	Spring	2018
FIN-201	Spring	2018

Note: CS-319 has two sections in Spring 2018

Query 1: Fall 2017 Courses (c1)

SQL Query

```sql
select course_id
from section
where semester = 'Fall' and year = 2017;
```

Before: Section Table

course_id	semester	year
CS-101	Fall	2017
CS-347	Fall	2017
PHY-101	Fall	2017
CS-101	Spring	2018
CS-319	Spring	2018
FIN-201	Spring	2018

After: Result (c1)

course_id
CS-101
CS-347
PHY-101

Analogy: Like filtering your closet for "Fall 2017" clothes only.

Query 2: Spring 2018 Courses (c2)

SQL Query

```sql
select course_id
from section
where semester = 'Spring' and year = 2018;
```

Before: Section Table

course_id	semester	year
CS-101	Fall	2017
CS-347	Fall	2017
PHY-101	Fall	2017
CS-101	Spring	2018
CS-319	Spring	2018
CS-319	Spring	2018
FIN-201	Spring	2018

After: Result (c2)

course_id
CS-101
CS-319
CS-319
FIN-201

Note

CS-319 appears *twice* because two sections were offered!

UNION: Combining Everything

What Does UNION Do?

Combines results from two queries, **removing duplicates**.

Party Analogy

- You're inviting friends from your *work party* OR *birthday party*
- If someone is on both lists, they get *one* invitation (no duplicates)
- UNION = Everyone invited to at least one party

UNION: The Query

SQL Query

```sql
(select course_id
from section
where semester = 'Fall' and year = 2017)
union
(select course_id
from section
where semester = 'Spring' and year = 2018);
```

- Parentheses are optional but improve readability
- Some databases don't allow parentheses
- **Automatically eliminates duplicates**

UNION: Before and After

Before: c1 and c2

c1 (Fall 2017)
CS-101
CS-347
PHY-101

c2 (Spring 2018)
CS-101
CS-319
CS-319
FIN-201

- CS-101: In both → appears once
- CS-319: Twice in c2 → appears once
- All courses from either semester

During union (with duplicates)

course_id
CS-101
CS-347
PHY-101
CS-101
CS-319
CS-319
FIN-201

After: UNION Result

course_id
CS-101
CS-347
PHY-101
CS-319
FIN-201

UNION ALL: Keeping Duplicates

SQL Query

```sql
(select course_id
from section
where semester = 'Fall' and year = 2017)
union all
(select course_id
from section
where semester = 'Spring' and year = 2018);
```

The Difference

UNION ALL keeps *all* duplicates from both queries.

- Total duplicates = duplicates in c1 + duplicates in c2
- CS-101: 1 (from c1) + 1 (from c2) = **2 times**
- CS-319: 0 (from c1) + 2 (from c2) = **2 times**

UNION ALL: Before and After

Before: c1 and c2

c1 (Fall 2017)
CS-101
CS-347
PHY-101

c2 (Spring 2018)
CS-101
CS-319
CS-319
FIN-201

After: UNION ALL Result

course_id
CS-101
CS-347
PHY-101
CS-101
CS-319
CS-319
FIN-201

Notice

7 rows total (3 from c1 + 4 from c2)

INTERSECT: Finding Common Ground

What Does INTERSECT Do?

Returns only the rows that appear in **both** queries.

Party Analogy

- You want to find friends invited to *both* your work party *and* birthday party
- Only people on both lists make the cut
- INTERSECT = VIP guests who made both lists

INTERSECT: The Query

SQL Query

```sql
(select course_id
from section
where semester = 'Fall' and year = 2017)
intersect
(select course_id
from section
where semester = 'Spring' and year = 2018);
```

- Finds courses offered in **both** semesters
- **Automatically eliminates duplicates**
- Result will only contain common elements

INTERSECT: Before and After

Before: c1 and c2

c1 (Fall 2017)
CS-101
CS-347
PHY-101

c2 (Spring 2018)
CS-101
CS-319
CS-319
FIN-201

After: INTERSECT Result

course_id
CS-101

Analysis

- Only CS-101 appears in both semesters
- Even though CS-319 appears twice in c2, it's not in c1
- Result: just one row

INTERSECT ALL: Keeping Duplicates

SQL Query

```sql
(select course_id
from section
where semester = 'Fall' and year = 2017)
intersect all
(select course_id
from section
where semester = 'Spring' and year = 2018);
```

The Rule

Number of duplicates in result = **minimum** of duplicates in c1 and c2.

Example:

- If ECE-101: 4 sections in Fall 2017, 2 sections in Spring 2018
- INTERSECT ALL result: min(4, 2) = **2 tuples** with ECE-101

INTERSECT ALL: Example Scenario

Before: Hypothetical Data

c1 (Fall 2017)
ECE-101
ECE-101
ECE-101
ECE-101
CS-101

c2 (Spring 2018)
ECE-101
ECE-101
CS-101

After: INTERSECT ALL

course_id
ECE-101
ECE-101
CS-101

Calculation

- ECE-101: min(4, 2) = 2
- CS-101: min(1, 1) = 1
- Total: 3 rows

EXCEPT: Set Difference

What Does EXCEPT Do?

Returns rows from the first query that are **not** in the second query.

Party Analogy

- Friends invited to your work party but *NOT* your birthday party
- "Work colleagues I won't see outside of work"
- EXCEPT = Exclusive to the first list

EXCEPT: The Query

SQL Query

```
(select course_id
from section
where semester = 'Fall' and year = 2017)
except
(select course_id
from section
where semester = 'Spring' and year = 2018);
```

- Finds courses in Fall 2017 **but not** in Spring 2018
- Performs set difference: c1 - c2
- **Automatically eliminates duplicates** before computing

EXCEPT: Before and After

Before: c1 and c2

c1 (Fall 2017)
CS-101
CS-347
PHY-101

c2 (Spring 2018)
CS-101
CS-319
CS-319
FIN-201

After: EXCEPT Result

course_id
CS-347
PHY-101

Analysis

- CS-101 removed (in both)
- CS-347: only in Fall $\rightarrow$ kept
- PHY-101: only in Fall $\rightarrow$ kept

EXCEPT ALL: Keeping Duplicates

SQL Query

```sql
(select course_id
from section
where semester = 'Fall' and year = 2017)
except all
(select course_id
from section
where semester = 'Spring' and year = 2018);
```

EXCEPT ALL: Example Scenario

Before: Hypothetical Data

c1 (Fall 2017)
ECE-101
ECE-101
ECE-101
ECE-101
CS-101

c2 (Spring 2018)
ECE-101
ECE-101
CS-101

After: EXCEPT ALL

course_id
ECE-101
ECE-101

Calculation

- ECE-101: 4 - 2 = 2
- CS-101: 1 - 1 = 0 (removed)
- Total: 2 rows

3.6 Null Values

What is NULL?

Analogy: NULL is like an empty box - you don't know what's inside (or if there's anything at all!)

Key Points:

- NULL represents *unknown* or *missing* data
- NULL $\neq$ 0, NULL $\neq$ empty string ''
- NULL is the "I don't know" answer in databases

Example:

- Salary = 50000 (we know it)
- Salary = NULL (we don't know it)

Arithmetic Operations with NULL

Analogy: If you add a mystery number to 5, the result is still a mystery!

Rule: Any arithmetic operation with NULL produces NULL

SQL Query:

```
SELECT name, salary, salary + 5000 AS new_salary
FROM instructor;
```

Before (instructor table):

name	salary
Einstein	95000
Gold	NULL
Katz	75000

After (Result):

name	salary	new_salary
Einstein	95000	100000
Gold	NULL	NULL
Katz	75000	80000

Notice: NULL + 5000 = NULL

Comparison Operations with NULL

Analogy: Is mystery box > 10? We can't say TRUE or FALSE... it's UNKNOWN!

Rule: Comparisons with NULL yield UNKNOWN (not TRUE or FALSE)

SQL Query:

```
SELECT name, salary
FROM instructor
WHERE salary > 80000;
```

Before (instructor table):

name	salary
Einstein	95000
Gold	NULL
Mozart	40000
Singh	80000

After (Result):

name	salary
Einstein	95000

Gold is excluded!

NULL > 80000 = UNKNOWN

Three-Valued Logic: TRUE, FALSE, UNKNOWN

SQL introduces a third logical value: UNKNOWN

AND Truth Table:

AND	T	F	U
T	T	F	U
F	F	F	F
U	U	F	U

Examples:

- TRUE AND UNKNOWN = UNKNOWN
- FALSE AND UNKNOWN = FALSE

NOT: NOT UNKNOWN = UNKNOWN

OR Truth Table:

OR	T	F	U
T	T	T	T
F	T	F	U
U	T	U	U

Examples:

- TRUE OR UNKNOWN = TRUE
- FALSE OR UNKNOWN = UNKNOWN

Testing for NULL Values

Analogy: You can't ask "Is mystery = mystery?" but you can ask "Is this a mystery?"

Use: IS NULL or IS NOT NULL

SQL Query:

```
SELECT name
FROM instructor
WHERE salary IS NULL;
```

Before (instructor table):

name	salary
Einstein	95000
Gold	NULL
Mozart	40000
Califieri	NULL

After (Result):

name
Gold
Califieri

Finds all instructors with unknown salaries

Testing for UNKNOWN Results

SQL Query:

```
SELECT name
FROM instructor
WHERE salary > 10000 IS UNKNOWN;
```

Before (instructor table):

name	salary
Einstein	95000
Gold	NULL
Mozart	5000
Singh	NULL

After (Result):

name
Gold
Singh

Logic:

- 95000 > 10000 = TRUE
- NULL > 10000 = UNKNOWN
- 5000 > 10000 = FALSE

NULL Values in DISTINCT

Analogy: When removing duplicates, two mystery boxes are treated as "the same mystery"

Rule: For DISTINCT, two NULLs are considered identical

SQL Query:

```
SELECT DISTINCT dept_name, salary
FROM instructor;
```

Before (instructor table):

name	dept	salary
Gold	Physics	NULL
Katz	Comp. Sci.	75000
Singh	Finance	NULL
Califieri	History	NULL

After (Result):

dept_name	salary
Physics	NULL
Comp. Sci.	75000
Finance	NULL
History	NULL

Note: All NULLs are kept because depts differ!

NULL in Set Operations

Rule: For UNION, INTERSECT, EXCEPT - NULLs are treated as identical **SQL**

Query:

```
SELECT salary FROM instructor
UNION
SELECT salary FROM student;
```

Before:

instructor:

salary
75000
NULL
NULL

student:

salary
NULL
50000

After (Result):

salary
75000
NULL
50000

All NULLs treated as one value, so only one NULL in result!

3.7 Aggregate Functions

What are Aggregate Functions?

Analogy: Aggregates are like a calculator that works on entire columns!

Five Standard Aggregate Functions:

1. **AVG()** - Average (mean) value
2. **MIN()** - Minimum value
3. **MAX()** - Maximum value
4. **SUM()** - Total sum
5. **COUNT()** - Number of values

Key Points:

- Take a *collection* of values as input
- Return a *single* value as output
- SUM and AVG require numbers
- MIN, MAX, COUNT work on any data type

AVG - Average Function

Analogy: Like calculating your grade point average!

SQL Query:

```
SELECT AVG(salary) AS avg_salary
FROM instructor
WHERE dept_name = 'Comp. Sci.';
```

Before (instructor table):

name	salary
Katz	75000
Srinivasan	65000
Brandt	92000
Einstein	95000
Mozart	40000

Only Comp. Sci. dept

After (Result):

avg_salary
77333.33

Calculation:

$$= (75000 + 65000 + 92000)/3$$
$$= 232000/3$$
$$= 77333.33$$

COUNT with DISTINCT

Analogy: Counting unique people in a room (not how many times you see them!)

SQL Query:

```
SELECT COUNT(DISTINCT ID) AS unique_instructors
FROM teaches
WHERE semester = 'Spring 2018';
```

Before (teaches table):

ID	course_id	semester
10101	CS-101	Spring 2018
10101	CS-315	Spring 2018
12121	FIN-201	Spring 2018
15151	MU-199	Fall 2017

ID 10101 teaches 2 courses

After (Result):

unique_instructors
2

Without DISTINCT: 3
With DISTINCT: 2

(10101 counted once!)

COUNT(*) - Counting Tuples

Analogy: Counting how many rows are in a table (like counting pages in a book)

SQL Query:

```
SELECT COUNT(*) AS total_courses
FROM course;
```

Before (course table):

course_id	title
CS-101	Intro to CS
CS-315	Robotics
FIN-201	Investment
MU-199	Music Video
PHY-101	Physics

After (Result):

total_courses
5

Note:
COUNT(*) counts all rows including NULL values!

MIN and MAX Functions

SQL Query:

```
SELECT MIN(salary) AS min_sal,
MAX(salary) AS max_sal
FROM instructor;
```

Before (instructor table):

name	salary
Einstein	95000
Katz	75000
Mozart	40000
Brandt	92000
Gold	NULL

After (Result):

min_sal	max_sal
40000	92000

MIN finds lowest value
MAX finds highest value

NULL values ignored!

SUM - Total Function

Analogy: Adding up all the money in your piggy bank!

SQL Query:

```
SELECT SUM(salary) AS total_salary
FROM instructor
WHERE dept_name = 'Comp. Sci.';
```

Before (instructor table):

name	salary
Katz	75000
Srinivasan	65000
Brandt	92000
Einstein	95000
Mozart	40000

After (Result):

total_salary
232000

Calculation:

$$= 75000 + 65000 + 92000$$
$$= 232000$$

GROUP BY - Grouping Data

Analogy: Like sorting students by their class year, then calculating average GPA for each year!

What does GROUP BY do?

1. Divides table into groups based on specified attributes
2. Computes aggregate function for each group separately
3. Returns one result row per group

Visual Example:

All Instructors $\rightarrow$ Group by Dept $\rightarrow$ One result per dept

Physics: Einstein, Gold, ...
Comp. Sci.: Katz, Brandt, ...
Finance: Singh, ...

GROUP BY Example

SQL Query:

```
SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor
GROUP BY dept_name;
```

Before (instructor table):

dept_name	salary
Comp. Sci.	75000
Comp. Sci.	65000
Physics	95000
Finance	80000
Comp. Sci.	92000

3 groups formed!

After (Result):

dept_name	avg_salary
Comp. Sci.	77333.33
Physics	95000.00
Finance	80000.00

One row per group
Average computed
within each group

GROUP BY with JOIN

SQL Query:

```
SELECT dept_name, COUNT(DISTINCT ID) AS instr_count
FROM instructor, teaches
WHERE instructor.ID = teaches.ID
AND semester = 'Spring' AND year = 2018
GROUP BY dept_name;
```

Before (after join):

dept_name	ID
Comp. Sci.	10101
Comp. Sci.	10101
Finance	12121
Music	15151

10101 appears twice (teaches 2 courses)

After (Result):

dept_name	instr_count
Comp. Sci.	1
Finance	1
Music	1

DISTINCT ensures 10101 counted once per department

GROUP BY - Important Rules

Core Rule: When using `GROUP BY`, every attribute in the `SELECT` clause must be either:

1. Listed in the `GROUP BY` clause, **OR**
2. Wrapped inside an aggregate function (e.g., `AVG()`, `COUNT()`, `SUM()`, `MAX()`)

Wrong Query (Invalid):

```
/* Error: ID is neither grouped nor aggregated */
SELECT dept_name, ID, AVG(salary)
FROM instructor
GROUP BY dept_name;
```

Why is this wrong? A department contains many instructors, each with a different `ID`. After grouping by `dept_name`, SQL does not know **which ID to show** for the group.

GROUP BY - Correct Usage

Correct Query:

```
SELECT dept_name, AVG(salary)
FROM instructor
GROUP BY dept_name;
```

Here, `dept_name` is in the GROUP BY clause, and `salary` is used inside an aggregate function is Valid.

HAVING Clause - Filtering Groups

Analogy:

- WHERE = filter individual students
- HAVING = filter entire classes

Key Differences:

	WHERE	**HAVING**
Applied to	Individual rows	Groups
When	Before grouping	After grouping
Can use aggregates?	No	Yes

Example:

- WHERE salary > 50000 $\rightarrow$ filters individual instructors
- HAVING AVG(salary) > 50000 $\rightarrow$ filters departments

HAVING Clause Example

SQL Query:

```
SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) > 42000;
```

Before (grouped data):

dept_name	avg_salary
Comp. Sci.	77333
Biology	72000
History	40000
Finance	85000
Music	40000

Pass HAVING test
Fail HAVING test

After (Result):

dept_name	avg_salary
Comp. Sci.	77333
Biology	72000
Finance	85000

Only departments with avg salary > 42000 are included

HAVING Clause - Important Rules

Rule: Attributes in HAVING must be either:

1. In the GROUP BY clause, OR
2. Inside an aggregate function

Wrong Query:

```
/* Erroneous - salary not in GROUP BY */
SELECT dept_name, AVG(salary)
FROM instructor
GROUP BY dept_name
HAVING salary > 50000;
```

Correct Query:

```
SELECT dept_name, AVG(salary)
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) > 50000;
```

Are GROUP BY and Aggregates Mutually Exclusive?

No. They are not mutually exclusive.

- You can use aggregates **without** GROUP BY.
- You can use GROUP BY **with** aggregates.
- You can even use GROUP BY **without aggregates**, but it's rarely useful.

Key Point: GROUP BY limits which columns you can select. Aggregates summarize data.

Example 1: Aggregate Without GROUP BY

Valid Query:

```
SELECT COUNT(*)
FROM instructor;
```

Why valid? The whole table becomes one group automatically.

Example 2: GROUP BY With Aggregate

Valid Query:

```
SELECT dept_name, AVG(salary)
FROM instructor
GROUP BY dept_name;
```

Why valid? Each group (department) gets its own average salary.

Example 3: GROUP BY Without Aggregate

Valid but uncommon:

```
SELECT dept_name
FROM instructor
GROUP BY dept_name;
```

Result: Shows unique department names.
(No aggregate needed here.)

Invalid: Non-grouped Column in SELECT

```
/* INVALID */
SELECT dept_name, salary
FROM instructor
GROUP BY dept_name;
```

Why invalid? Multiple salaries exist inside one group. SQL cannot choose one salary.

Correct Version

```
SELECT dept_name, AVG(salary)
FROM instructor
GROUP BY dept_name;
```

salary is now summarized -> allowed.

HAVING Works Only With Groups

Invalid:

```
HAVING salary > 50000
```

Correct:

```
HAVING AVG(salary) > 50000
```

HAVING filters **groups**, not individual rows.

SQL Query Processing Order

Analogy: Like an assembly line - each step processes what the previous step produced!

Processing Order:

1. **FROM** - Get the table(s)
2. **WHERE** - Filter individual rows
3. **GROUP BY** - Create groups
4. **HAVING** - Filter groups
5. **SELECT** - Choose columns and compute aggregates

Example Flow:

instructor table $\rightarrow$ WHERE filter $\rightarrow$ GROUP BY dept $\rightarrow$
HAVING filter $\rightarrow$ SELECT dept, AVG(salary)

WHERE and HAVING Together

SQL Query:

```
SELECT course_id, semester, year, sec_id,
AVG(tot_cred) AS avg_cred
FROM student, takes
WHERE student.ID = takes.ID AND year = 2017
GROUP BY course_id, semester, year, sec_id
HAVING COUNT(ID) >= 2;
```

Before (after WHERE):

course	sem	yr	cred
CS-101	Fall	17	80
CS-101	Fall	17	90
CS-101	Fall	17	70
FIN-201	Spr	17	85

CS-101: 3 students
FIN-201: 1 student

After (Result):

course	sem	yr	sec	avg
CS-101	Fall	17	1	80

Only sections with 2+ students shown

FIN-201 filtered out by HAVING clause

Aggregate Functions and NULL

Analogy: Aggregates ignore mystery boxes (NULLs) when calculating!

SQL Query:

```
SELECT SUM(salary) AS total_salary,
AVG(salary) AS avg_salary,
COUNT(*) AS total_rows,
COUNT(salary) AS non_null_salaries
FROM instructor;
```

Before (instructor):

name	salary
Einstein	95000
Gold	NULL
Katz	75000
Mozart	NULL
Singh	80000

After (Result):

total	avg	rows	non-null
250000	83333	5	3

SUM: 95k+75k+80k
AVG: 250k/3 (not 5!)
COUNT(*): all 5 rows
COUNT(salary): only 3

Aggregate Functions - NULL Handling Rules

Critical Rule: All aggregates except COUNT(*) ignore NULL values

Function	NULL Handling	Empty Set
COUNT(*)	Counts all rows	Returns 0
COUNT(attr)	Ignores NULLs	Returns 0
SUM(attr)	Ignores NULLs	Returns NULL
AVG(attr)	Ignores NULLs	Returns NULL
MIN(attr)	Ignores NULLs	Returns NULL
MAX(attr)	Ignores NULLs	Returns NULL

Key Insight:

- COUNT(*) = number of rows (including NULLs)
- COUNT(salary) = number of non-NULL salaries
- Empty collection → COUNT gives 0, others give NULL

Aggregates on Empty Collections

SQL Query:

```
SELECT COUNT(*) AS count_all,
COUNT(salary) AS count_sal,
SUM(salary) AS total,
AVG(salary) AS average
FROM instructor
WHERE dept_name = 'Archaeology';
```

Before (instructor):

dept_name	salary
Comp. Sci.	75000
Physics	95000
Finance	80000

No Archaeology dept!
Empty result after WHERE

After (Result):

count_all	count_sal	total	avg
0	0	NULL	NULL

COUNT returns 0
Others return NULL

Can't average nothing!

Boolean Aggregates: SOME and EVERY

Introduced in SQL:1999

SQL Query:

```
SELECT dept_name,
EVERY(salary > 50000) AS all_high_paid,
SOME(salary > 80000) AS any_very_high
FROM instructor
GROUP BY dept_name;
```

Before (instructor):

dept_name	salary
Comp. Sci.	75000
Comp. Sci.	92000
Finance	85000
Finance	45000

After (Result):

dept	all_high	any_very
Comp. Sci.	TRUE	TRUE
Finance	FALSE	TRUE

Logic:

- EVERY = AND all values
- SOME = OR all values

Example with WHERE and HAVING

```
SELECT dept_name, AVG(salary)
FROM instructor
WHERE salary > 30000
GROUP BY dept_name
HAVING AVG(salary) > 60000;
```

WHERE removes low-salary rows. **HAVING** removes low-average groups.

Common Mistakes to Avoid

1: Using = NULL instead of IS NULL

- Wrong: WHERE salary = NULL (Always returns no rows!)
- Correct: WHERE salary IS NULL

2: Forgetting DISTINCT with COUNT

- COUNT(ID) counts all occurrences
- COUNT(DISTINCT ID) counts unique IDs

3: Non-aggregated attributes in SELECT

- Must be in GROUP BY or inside aggregate function
- Each group needs unambiguous values

Common Mistakes to Avoid (cont.)

4: Using aggregates in WHERE

- Wrong: WHERE AVG(salary) > 50000
- Correct: HAVING AVG(salary) > 50000

5: Assuming NULL = 0

- NULL means unknown, not zero
- SUM ignores NULLs (doesn't treat as 0)
- COUNT(*) includes NULLs, COUNT(attr) excludes them

6: Wrong ORDER of clauses

- Correct: SELECT - FROM - WHERE - GROUP BY - HAVING
- Cannot put HAVING before GROUP BY

3.8 Nested Subqueries

What Are Nested Subqueries?

SQL lets you put queries inside queries!

- A **subquery** is a SELECT statement nested inside another query
- The inner query runs first, produces results
- The outer query uses those results
- Think of it as: "Answer this question, then use that answer to solve the bigger question"

Where can subqueries appear?

- WHERE clause (most common)
- FROM clause
- SELECT clause
- HAVING clause

3.8.1 Set Membership: IN

Analogy: Guest List

"Is your name IN the VIP list?" - Check if something belongs to a set

Query: Find courses taught in BOTH Fall 2017 AND Spring 2018

```
SELECT DISTINCT course_id
FROM section
WHERE semester = 'Fall' AND year = 2017
AND course_id IN
(SELECT course_id
FROM section
WHERE semester = 'Spring' AND year = 2018);
```

Logic: "Show me Fall 2017 courses that also appear in the Spring 2018 list"

IN Operator: Before & After

Before: section table

course_id	semester	year
CS-101	Fall	2017
CS-347	Fall	2017
CS-101	Spring	2018
PHY-101	Fall	2017
MUS-201	Spring	2018
CS-347	Spring	2018

Subquery returns:

CS-101
MUS-201
CS-347

After: Query Result

course_id
CS-101
CS-347

Explanation:

- Inner query finds Spring 2018 courses
- Outer query finds Fall 2017 courses
- IN checks which Fall courses are in Spring list
- Only CS-101 and CS-347 appear in both!

Set Membership: NOT IN

Analogy: Exclusion List

"Show me everyone NOT IN the banned list" - Find what's NOT in a set

Query: Find courses in Fall 2017 but NOT in Spring 2018

```sql
SELECT DISTINCT course_id
FROM section
WHERE semester = 'Fall' AND year = 2017
AND course_id NOT IN
(SELECT course_id
FROM section
WHERE semester = 'Spring' AND year = 2018);
```

Similar to EXCEPT operation, but written differently!

NOT IN: Before & After

Before: section table

course_id	semester	year
CS-101	Fall	2017
CS-347	Fall	2017
PHY-101	Fall	2017
BIO-201	Fall	2017
CS-101	Spring	2018
MUS-201	Spring	2018
CS-347	Spring	2018

After: Query Result

course_id
PHY-101
BIO-201

Explanation:

- Find all Fall 2017 courses
- Exclude those that appear in Spring 2018
- PHY-101 and BIO-201 only in Fall!

IN with Enumerated Sets

You can also use IN with explicit lists:

```
SELECT DISTINCT name
FROM instructor
WHERE name NOT IN ('Mozart', 'Einstein');
```

Before: instructor

name
Einstein
Mozart
Beethoven
Chopin

After: Result

name
Beethoven
Chopin

Simple filtering!

3.8.2 Set Comparison: SOME

Analogy: Beating at Least One Player

"I scored higher than SOME players" - you beat at least one person

Query: Find instructors earning more than at least one Biology dept instructor

```
SELECT name
FROM instructor
WHERE salary > SOME
(SELECT salary
FROM instructor
WHERE dept_name = 'Biology');
```

Operators: >, <, >=, <=, =, <> with SOME

Note: = SOME is identical to IN!

SOME: Before & After

Before: instructor

name	dept	salary
Einstein	Physics	95000
Mozart	Music	40000
Darwin	Biology	72000
Watson	Biology	68000
Brandt	Comp Sci	92000
Kim	Elec Eng	80000

Subquery returns: {72000, 68000}

After: Result

name
Einstein
Darwin
Brandt
Kim

Logic:

- Biology salaries: 72K, 68K
- Find anyone > 68K OR > 72K
- Mozart (40K) doesn't qualify
- Everyone else earns more than at least one Biology instructor

Set Comparison: ALL

Analogy: Champion of Everyone

"I scored higher than ALL players" - you beat everyone!

Query: Find instructors earning more than EVERY Biology instructor

```
SELECT name
FROM instructor
WHERE salary > ALL
(SELECT salary
FROM instructor
WHERE dept_name = 'Biology');
```

Key Difference:

- SOME = at least one (*any*)
- ALL = every single one (*all*)

Note: <> ALL is identical to NOT IN!

ALL: Before & After

Before: instructor

name	dept	salary
Einstein	Physics	95000
Mozart	Music	40000
Darwin	Biology	72000
Watson	Biology	68000
Brandt	Comp Sci	92000
Kim	Elec Eng	80000

Subquery returns: {72000, 68000}

After: Result

name
Einstein
Brandt
Kim

Logic:

- Must beat HIGHEST Biology salary (72K)
- Einstein: 95K > 72K ✓
- Brandt: 92K > 72K ✓
- Kim: 80K > 72K ✓
- Mozart: 40K < 72K ✗
- Darwin: is Biology ✗

Practical Example: Finding Maximum

Query: Find departments with highest average salary

```
SELECT dept_name
FROM instructor
GROUP BY dept_name
HAVING AVG(salary) >= ALL
(SELECT AVG(salary)
FROM instructor
GROUP BY dept_name);
```

Step-by-step:

1. Inner query: compute all department average salaries
2. Outer query: find departments whose average >= ALL others
3. Result: department(s) with maximum average salary

3.8.3 Test for Empty Relations: EXISTS

Analogy: Checking if Anyone's Home

"Is there anyone inside?" - EXISTS checks if a result set has any rows

Query: Find courses taught in both Fall 2017 and Spring 2018

```
SELECT course_id
FROM section AS S
WHERE semester = 'Fall' AND year = 2017
AND EXISTS
(SELECT *
FROM section AS T
WHERE semester = 'Spring' AND year = 2018
AND S.course_id = T.course_id);
```

This is a CORRELATED subquery - uses variable S from outer query!

EXISTS: Before & After

Before: section

course_id	sem	year
CS-101	Fall	2017
CS-347	Fall	2017
PHY-101	Fall	2017
CS-101	Spring	2018
CS-347	Spring	2018
BIO-201	Spring	2018

How it works: For each Fall 2017 course, check if a matching Spring 2018 entry EXISTS

After: Result

course_id
CS-101
CS-347

Process:

- CS-101: Check Spring 2018... EXISTS!
- CS-347: Check Spring 2018... EXISTS!
- PHY-101: Check Spring 2018... doesn't exist

NOT EXISTS: Checking Non-Existence

Analogy: Division Check (Superset)

"Did the student take ALL required courses?" - Check if nothing is missing

Query: Find students who took ALL Biology courses

```
SELECT S.ID, S.name
FROM student AS S
WHERE NOT EXISTS
((SELECT course_id
FROM course
WHERE dept_name = 'Biology')
EXCEPT
(SELECT T.course_id
FROM takes AS T
WHERE S.ID = T.ID));
```

Logic: "Biology courses" EXCEPT "courses taken" = empty set?

NOT EXISTS: Before & After

Biology Courses:

BIO-101
BIO-201
BIO-301

Student 12345 took:

BIO-101
BIO-201
BIO-301
CS-101

Student 67890 took:

BIO-101
BIO-201

After: Result

ID	name
12345	Smith

Check:

- Student 12345:
  {BIO-101, BIO-201, BIO-301}
  EXCEPT {all taken} = {}
  NOT EXISTS({}) = TRUE
- Student 67890:
  {BIO-101, BIO-201, BIO-301}
  EXCEPT {only 2} = {BIO-301}
  NOT EXISTS({BIO-301}) = FALSE

3.8.4 Test for Duplicates: UNIQUE

Analogy: Attendance Check

"Was this course offered once or multiple times?" - Check for duplicates

Query: Find courses offered at most once in 2017

```
SELECT T.course_id
FROM course AS T
WHERE UNIQUE
(SELECT R.course_id
FROM section AS R
WHERE T.course_id = R.course_id
AND R.year = 2017);
```

UNIQUE returns TRUE if:

- No duplicates exist, OR
- Result is empty (course not offered = at most once!)

Test for Duplicates: UNIQUE

The Logic

Query: Find courses offered *at most once* in 2017.

- **UNIQUE** returns TRUE if the subquery result contains **no duplicates**.
- **Important:** An empty set (0 rows) has no duplicates!

1. The Data (Section Table)

course_id	year
CS-101	2017
CS-101	2017
BIO-201	2017

**Note:*

MUS-101 is not in this table.

2. The Execution (Row by Row)

Course	Subquery Found	UNIQUE?
CS-101	{CS-101, CS-101}	**FALSE**
BIO-201	{BIO-201}	**TRUE**
MUS-101	∅ **(Empty Set)**	**TRUE**

Final Result: `BIO-201, MUS-101`

NOT UNIQUE: Finding Duplicates

Query: Find courses offered at least twice in 2017

```
SELECT T.course_id
FROM course AS T
WHERE NOT UNIQUE
(SELECT R.course_id
FROM section AS R
WHERE T.course_id = R.course_id
AND R.year = 2017);
```

Same data as before

Result: Opposite!

course_id
CS-101

Only CS-101 appears multiple times

3.8.5 Subqueries in FROM Clause

Analogy: Creating a Summary Report First

"Calculate departmental averages, then filter those averages"

Query: Find departments with average salary > 42000

```
SELECT dept_name, avg_salary
FROM (SELECT dept_name, AVG(salary) AS avg_salary
FROM instructor
GROUP BY dept_name)
WHERE avg_salary > 42000;
```

The subquery creates a temporary table we can query!

- Inner query: creates aggregated summary
- Outer query: filters that summary
- No HAVING clause needed!

FROM Subquery: Before & After

Before: instructor

dept	salary
Comp Sci	92000
Comp Sci	75000
Biology	72000
Biology	68000
Music	40000
Music	35000

Subquery creates:

dept	avg_salary
Comp Sci	83500
Biology	70000
Music	37500

After: Result

dept	avg_salary
Comp Sci	83500
Biology	70000

Process:

1. Inner: GROUP BY and compute averages
2. Result: temporary table with 3 rows
3. Outer: Filter where avg_salary > 42000
4. Music (37500) eliminated!

Naming Subquery Results

You can name the temporary relation and its attributes:

```
SELECT dept_name, avg_salary
FROM (SELECT dept_name, AVG(salary)
FROM instructor
GROUP BY dept_name)
AS dept_avg(dept_name, avg_salary)
WHERE avg_salary > 42000;
```

Benefits:

- Makes query more readable
- Required in MySQL and PostgreSQL
- Can reference the names in outer query

Database differences:

- MySQL/PostgreSQL: MUST name subqueries
- Oracle: Can name but cannot rename attributes

Complex FROM Example: Finding Maximum

Query: Find maximum total salary across all departments

```
SELECT MAX(tot_salary)
FROM (SELECT dept_name, SUM(salary)
FROM instructor
GROUP BY dept_name)
AS dept_total(dept_name, tot_salary);
```

Subquery produces:

dept	tot_salary
Comp Sci	245000
Biology	180000
Music	95000

Final Result:

MAX
245000

Two-step process:

1. Sum salaries per dept
2. Find maximum sum

3.8.6 The WITH Clause

Mental Model: Temporary Tables

Think of `WITH` as creating a **temporary table** that exists only for the duration of your query.

Goal: Find the department(s) with the highest budget.

```sql
WITH max_budget(value) AS (
-- Step 1: Find the single highest number
SELECT MAX(budget)
FROM department
)
-- Step 2: Use that number to find the department
SELECT dept_name, budget
FROM department, max_budget
WHERE department.budget = max_budget.value;
```

Why use this?

3.8.6 The WITH Clause (cont.)

- Breaks complex logic into small steps.
- Much easier to read than nested subqueries.

Visualizing the Process

Step 1: Create the Temporary Table

Source: Department

dept_name	budget
Comp Sci	120000
Biology	90000
Music	80000
Physics	120000

$\Rightarrow$

Temp: max_budget

value
120000

Step 2: Match and Filter

- The query looks for rows where `budget = 120000`.

Final Result:

dept_name	budget
Comp Sci	120000
Physics	120000

Multiple WITH Clauses

Goal: Find departments where the total salary is higher than the average across all departments.
This requires a chain of logic:

1. Calculate total salary per department.
2. Calculate the average of those totals.
3. Compare specific departments to that average.

```sql
WITH
-- Table 1: Sum salaries by department
dept_total(dept_name, value) AS (
SELECT dept_name, SUM(salary)
FROM instructor
GROUP BY dept_name
),  -- Note the comma here!
```

Multiple WITH Clauses (cont.)

```sql
-- Table 2: Find average of Table 1
dept_total_avg(value) AS (
SELECT AVG(value)
FROM dept_total
)

-- Final Query: Compare Table 1 vs Table 2
SELECT dept_name
FROM dept_total, dept_total_avg
WHERE dept_total.value > dept_total_avg.value;
```

Logic Flow: The Chain Reaction

1. dept_total
(Sum of salaries)

Dept	Sum
CS	240k
Bio	180k
Music	80k
Phys	200k

2. dept_total_avg
(Average of Col 1)

Avg
175k

⇕ Compare

3. Final Result
(Where Sum > Avg)

Result
CS
Bio
Phys

Explanation:

- We calculated the cutoff point (175k) in Step 2.
- **Music (80k)** is less than 175k, so it is excluded.
- The others are included.

3.8.7 Scalar Subqueries

Analogy: Inserting a Single Value

Like a function that returns one number to use in a calculation

Query: List departments with their instructor count

```
SELECT dept_name,
(SELECT COUNT(*)
FROM instructor
WHERE department.dept_name =
instructor.dept_name)
AS num_instructors
FROM department;
```

Scalar subquery: Returns exactly ONE value (single row, single column)

- Can appear in SELECT, WHERE, HAVING
- Must return only 1 tuple with 1 attribute
- Runtime error if multiple tuples returned!

Scalar Subquery: Before & After

Before: department

dept_name
Comp Sci
Biology
Music
Physics

instructor table:

name	dept
Einstein	Physics
Mozart	Music
Brandt	Comp Sci
Kim	Elec Eng
Gold	Physics
Singh	Comp Sci

After: Result

dept_name	num_instr
Comp Sci	2
Biology	0
Music	1
Physics	2

For each department:

- Subquery counts instructors
- Returns single number
- Displayed in result

Scalar Subqueries: Key Points

Where can scalar subqueries appear?

- SELECT clause (as shown)
- WHERE clause (for comparisons)
- HAVING clause (for filtering groups)

Example in WHERE:

```
SELECT name
FROM instructor
WHERE salary > (SELECT AVG(salary)
FROM instructor);
```

Important:

- Aggregate functions (COUNT, AVG, MAX) naturally return one value
- Without aggregates, be careful - might return multiple rows!
- SQL implicitly extracts the value from the single tuple

3.8.8 Scalar Without FROM Clause

Analogy: Pure Calculation

Sometimes you just need to calculate something without querying a table

Query: Average sections per instructor

```
SELECT (SELECT COUNT(*) FROM teaches) /
(SELECT COUNT(*) FROM instructor)
FROM dual;
```

Why "dual"?

- Some databases require a FROM clause
- **dual** is a dummy table with one row
- Oracle provides it; you can create it in other systems
- Some modern databases don't need FROM for calculations

Scalar Without FROM: Example

Data we're using:
teaches: 150 tuples

ID	course
10101	CS-101
10101	CS-347
12121	FIN-201
...	...

instructor: 50 tuples

ID
10101
12121
...

Result:

average
3

Calculation:

- COUNT(*) from teaches = 150
- COUNT(*) from instructor = 50
- Division: 150 / 50 = 3
- Average: 3 sections per instructor

Note: Integer division!
Multiply by 1.0 for decimals:

```
1.0 * COUNT(*) / COUNT(*)
```

Summary: Subquery Types

Type	Location	Purpose
Set Membership	WHERE	IN, NOT IN
Set Comparison	WHERE	SOME, ALL
Existence Test	WHERE	EXISTS, NOT EXISTS
Uniqueness Test	WHERE	UNIQUE, NOT UNIQUE
Derived Table	FROM	Create temp table
Temp Relation	WITH	Define once, use many
Scalar	SELECT/WHERE	Single value return

Key Concepts:

- **Correlated:** Inner query uses outer query variables
- **Uncorrelated:** Inner query independent
- **Scalar:** Returns exactly one value

Comparison Table: Different Approaches

Same question, multiple ways!

Approach	Query Style
INTERSECT	`(SELECT ...) INTERSECT (SELECT ...)`
IN	`WHERE course_id IN (SELECT ...)`
EXISTS	`WHERE EXISTS (SELECT ... AND S.id = T.id)`

Comparison	Equivalent
= SOME	IN
<> ALL	NOT IN
EXISTS	(often similar to IN)
NOT EXISTS with EXCEPT	Division / "for all"

SQL offers flexibility - choose what's most natural!

3.9 Modification of the Database

Deletion: Basics

Core Concepts:

- Deletes **whole tuples** (rows), not specific column values.
- Operates on **one relation** at a time.
- Syntax: `delete from r where P;`

The Danger of Omission:

```
delete from instructor;
```

Warning: If the `where` clause is omitted, **all** tuples are deleted. The table structure (schema) remains, but it is empty.

Deletion: Simple Predicates

Match Value:

```
delete from instructor
where dept_name = 'Finance';
```

Removes all Finance instructors.

Match Range:

```
delete from instructor
where salary between
13000 and 15000;
```

Removes instructors within salary band.

Deletion: Nested Queries

Scenario: Delete instructors located in the 'Watson' building. (Requires checking the `department` table first).

```
delete from instructor
where dept_name in (
select dept_name
from department
where building = 'Watson'
);
```

This finds the departments first, then deletes the matching instructors.

Deletion: Execution Order Important

Scenario: Delete instructors with salary below average.

```
delete from instructor
where salary < (
select avg(salary)
from instructor
);
```

Crucial Concept:

- The `avg(salary)` is calculated **once** before any deletions occur.
- If tuples were deleted one-by-one while recalculating the average, the result would depend on the processing order (incorrect behavior).

Insertion: Single Tuple Methods

1. Positional (Implicit Order):

```
insert into course
values ('CS-437', 'DB Systems', 'Comp. Sci.', 4);
```

Dangerous if schema order changes.

2. Named Attributes (Explicit Order):

```
insert into course (course_id, title, dept_name, credits)
values ('CS-437', 'DB Systems', 'Comp. Sci.', 4);
```

Safer. Allows changing order of arguments.

Insertion: Based on Queries

Scenario: Make Music students (>144 credits) into Instructors.

```
insert into instructor
select ID, name, dept_name, 18000
from student
where dept_name = 'Music'
and tot_cred > 144;
```

Concept:

- Instead of `values(...)`, we use a `select` statement.
- The select is evaluated fully before any insertion begins.

Insertion: The Infinite Loop Risk

Consider this request without a Primary Key:

```
insert into student
select * from student;
```

The Problem:

1. If the database inserts matches *while* selecting...
2. It reads a tuple, inserts a copy.
3. It reads the copy, inserts a third copy.
4. Result: Infinite loop of duplication.

Resolution: SQL standard requires the `select` to complete fully before `insert` starts.

Result of Self-Insertion

Query:

```
insert into student
select * from student;
```

Scenario A: No Primary Key

- The database successfully duplicates every tuple.
- **Result:** The table size doubles ($N \to 2N$ rows).

Scenario B: With Primary Key (Standard)

- The database attempts to insert duplicates of existing IDs.
- **Result:** Integrity Constraint Violation Error.
- The transaction aborts; no data changes.

Insertion: Handling Nulls

Explicit Null:

```
insert into student
values ('3003', 'Green', 'Finance', null);
```

Implicit Null: If an attribute is omitted from the named list, it defaults to null.

```
insert into student (ID, name, dept_name)
values ('3003', 'Green', 'Finance');
```

(Here, `tot_cred` becomes null automatically).

Insertion: Bulk Loading

Concept:

- Inserting thousands of rows using `insert into` statements is slow.
- **Bulk Loader Utilities:** Special database tools designed to read formatted text files (CSV, TSV).
- **Benefit:** Executes much faster than equivalent sequences of individual insert statements.

Updates: Basic & Nested

Unconditional (All Rows):

```
update instructor
set salary = salary * 1.05;
```

Nested Subquery: Give raise to those earning less than average.

```
update instructor
set salary = salary * 1.05
where salary < (select avg(salary) from instructor);
```

Like deletion, the average is calculated once at the start.

Updates: The Order Problem

Goal:

- Salary > 100,000 gets **3%** raise.
- Salary $\leq$ 100,000 gets **5%** raise.

Incorrect Approach:

```
update instructor set salary = salary * 1.03
where salary > 100000; -- Update 1

update instructor set salary = salary * 1.05
where salary <= 100000; -- Update 2
```

Issue: An instructor with 99,000 gets a 5% raise (to 103,950) in Update 2. If Update 2 ran first, they might accidentally get the 3% raise afterwards too! Order matters.

Updates: The CASE Solution

Concept: Performs both checks simultaneously on the *original* values.

```
update instructor
set salary = case
when salary <= 100000 then salary * 1.05
else salary * 1.03
end;
```

General Syntax: Returns the result of the first satisfied predicate.

The CASE Construct: General Syntax

Purpose: Acts like an if-then-else logic within an SQL statement. It returns the result of the **first** satisfied predicate.

General Form:

```
case
when pred1 then result1
when pred2 then result2
...
when predn then resultn
else result0
end
```

- `pred`: A condition (e.g., `salary < 50000`)
- `result`: The value returned if true.
- `else`: Returned if *none* of the predicates are true.

Updates: Scalar Subqueries

Scenario: Update `student.tot_cred` by summing data from `takes` table.

```
update student S
set tot_cred = (
select sum(credits)
from takes T, course C
where T.course_id = C.course_id
and S.ID = T.ID
and T.grade <> 'F'
and T.grade is not null
);
```

Updates: Handling Null Aggregates (Method 1)

Problem: If a student has taken no courses, `sum(credits)` returns **null**, not 0.

Solution 1 (CASE):

```
select case
when sum(credits) is not null
then sum(credits)
else 0
end
```

Updates: Handling Null Aggregates (Method 2)

Solution 2 (Coalesce): Many systems support the `coalesce` function. It returns the first non-null value in the list.

```
-- Replaces the subquery select clause
select coalesce(sum(credits), 0)
...
```

This is more concise than the `case` expression.

Questions?